

Informe Triggers Módulo 4

Modelo de Predicción

Versión: 1.2

19 de mayo de 2026

Juan David Peña Roldán

Universidad Surcolombiana

Neiva, Huila

2026

TGR-M04-01:

trg_versiones_modelos_inmutabilidad

Descripción	Impide cualquier UPDATE o DELETE sobre registros de versiones de modelos una vez persistidos. Lanza excepción ante cualquier intento de UPDATE o DELETE sobre <code>modulo4.versiones_modelos</code> .
Aplicado a	RF-69
Restricción origen RF	<i>"Inmutabilidad del Historial: Todas las versiones y transiciones se almacenan en esquema append-only. No se permite modificación ni eliminación." — RF-69, Restricción 7. También CA-8: "El sistema no permite UPDATE ni DELETE sobre ningún registro de versión o transición de estado."</i>
Justificación	Un ORM, un script de migración o un error de backend podría alterar el historial. La BD es la última línea de defensa. Esta garantía no puede delegarse a la capa de aplicación en un sistema de auditoría clínica.

```
CREATE OR REPLACE FUNCTION
modulo4.fn_versiones_modelos_inmutabilidad()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'DELETE' THEN
    RAISE EXCEPTION
      '[TGR-M04-01] Las versiones de modelos son inmutables
      (append-only). '
      'No se permite eliminar id_version_modelo=%. RF-69,
      Restricción 7.',
      OLD.id_version_modelo;
  END IF;

  IF TG_OP = 'UPDATE' THEN
    IF OLD.nombre_version      IS DISTINCT FROM
      NEW.nombre_version
```

```

        OR OLD.algoritmo                IS DISTINCT FROM
NEW.algoritmo
        OR OLD.tipo_modelo              IS DISTINCT FROM
NEW.tipo_modelo
        OR OLD.hash_artefacto           IS DISTINCT FROM
NEW.hash_artefacto
        OR OLD.hash_artefacto_sha256    IS DISTINCT FROM
NEW.hash_artefacto_sha256
        OR OLD.dataset_entrenamiento_hash IS DISTINCT FROM
NEW.dataset_entrenamiento_hash
        OR OLD.fecha_entrenamiento      IS DISTINCT FROM
NEW.fecha_entrenamiento
        OR OLD.f1_score                 IS DISTINCT FROM
NEW.f1_score
        OR OLD.recall_modelo            IS DISTINCT FROM
NEW.recall_modelo
        OR OLD.recall_clase_riesgo_alto IS DISTINCT FROM
NEW.recall_clase_riesgo_alto
        OR OLD.accuracy                 IS DISTINCT FROM
NEW.accuracy
        OR OLD.precision_modelo         IS DISTINCT FROM
NEW.precision_modelo
        OR OLD.auc_roc                  IS DISTINCT FROM NEW.auc_roc
        OR OLD.roc_auc_score            IS DISTINCT FROM
NEW.roc_auc_score
        OR OLD.id_proceso_rf71          IS DISTINCT FROM
NEW.id_proceso_rf71
        OR OLD.formato_artefacto        IS DISTINCT FROM
NEW.formato_artefacto
        OR OLD.tamano_artefacto_bytes   IS DISTINCT FROM
NEW.tamano_artefacto_bytes
        OR OLD.compatibilidad_variables IS DISTINCT FROM
NEW.compatibilidad_variables
        OR OLD.version_referencia       IS DISTINCT FROM
NEW.version_referencia
        OR OLD.matriz_confusion         IS DISTINCT FROM
NEW.matriz_confusion
        OR OLD.recall_por_clase         IS DISTINCT FROM
NEW.recall_por_clase
    THEN
        RAISE EXCEPTION
        '[TGR-M04-01] Los campos estructurales de una versión de

```

```

modelo son inmutables. '
    'id_version_modelo=%. Solo se permite modificar:
estado_version, notas_validacion, '
    'esta_produccion, fecha_despliegue, fecha_retiro. RF-69,
Restricción 7.',
    OLD.id_version_modelo;
END IF;
END IF;

RETURN NEW;
END;
$$;

```

Nota: El trigger original bloqueaba todo **UPDATE**. Con TGR-M04-26, necesitamos permitir **UPDATE** del campo **estado_version** (que es gestionado por TGR-M04-05 y TGR-M04-26), y del campo **notas_validacion** (RF-69, Fase 4: el veterinario puede agregar notas a un modelo aprobado antes de activarlo), y **esta_produccion** (campo legacy que sincroniza TGR-M04-05). Todo lo demás sigue siendo inmutable.

TGR-M04-02:

trg_predicciones_inmutabilidad

Descripción	Impide cualquier UPDATE o DELETE sobre registros de predicciones (resultados de inferencia) una vez persistidos. Lanza excepción ante cualquier intento de UPDATE o DELETE sobre modulo4.predicciones .
Aplicado a	RF-66, RF-67
Restricción origen RF	<i>"Los resultados de inferencia son inmutables una vez persistidos." — RF-66, Restricción 5. Y RF-67: "Bajo ninguna circunstancia se permite la edición o borrado de un resultado de inferencia una vez persistido."</i>
Justificación	Las predicciones son evidencia clínica y médico-legal. La inmutabilidad debe ser

	garantizada a nivel de motor de base de datos, no solo por políticas de aplicación.
--	---

```
CREATE OR REPLACE FUNCTION modulo4.fn_predicciones_inmutabilidad()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M04-02] Los resultados de inferencia son inmutables
(append-only). '
            'No se permite eliminar id_prediccion=%. '
            'Requerimiento RF-66 Restricción 5, RF-67.',
            OLD.id_prediccion;
    END IF;

    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            '[TGR-M04-02] Los resultados de inferencia son inmutables
(append-only). '
            'No se permite modificar id_prediccion=%. '
            'Requerimiento RF-66 Restricción 5, RF-67.',
            OLD.id_prediccion;
    END IF;

    RETURN NULL;
END;
$$;

CREATE OR REPLACE TRIGGER trg_predicciones_inmutabilidad
BEFORE UPDATE OR DELETE
ON modulo4.predicciones
FOR EACH ROW
EXECUTE FUNCTION modulo4.fn_predicciones_inmutabilidad();
```

TGR-M04-03:

trg_alertas_patologicas_inmutabilidad

Descripción	
	Impide el DELETE sobre alertas patológicas. Permite UPDATE únicamente

	sobre campos de gestión operativa (<i>estado_alerta</i>, <i>fecha_notificacion</i>, <i>diagnostico_confirmado</i>, <i>es_verdadero</i>, <i>id_usuario</i>) pero bloquea modificación de campos estructurales del resultado. Bloquea DELETE. En UPDATE bloquea modificación de campos inmutables: <i>id_prediccion</i>, <i>id_activo_biologico</i>, <i>id_finca</i>, <i>id_patologia</i>, <i>probabilidad_pct</i>, <i>nivel_criticidad</i>, <i>fecha_generacion</i>.
Aplicado a	RF-66, RF-67, RF-73
Restricción origen RF	RF-67: <i>"El historial es de solo lectura. No permite modificación de los resultados de inferencia una vez persistidos."</i> RF-73: <i>"Ningún registro de auditoría puede ser modificado o eliminado una vez persistido."</i>
Justificación	Las alertas son parte del historial clínico y de auditoría. Los campos de origen de la alerta (probabilidad, patología, nivel de criticidad, fecha de generación) no deben poder alterarse. Solo el ciclo de vida operativo de la alerta (gestión por el usuario) puede cambiar.

```

CREATE OR REPLACE FUNCTION
modulo4.fn_alertas_patologicas_inmutabilidad()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M04-03] No se permite eliminar alertas patológicas. '
            'id_alerta_patologica=%. Requerimiento RF-67, RF-73.',
            OLD.id_alerta_patologica;
    END IF;

    IF TG_OP = 'UPDATE' THEN
        IF OLD.id_prediccion      IS DISTINCT FROM NEW.id_prediccion
OR
           OLD.id_activo_biologico IS DISTINCT FROM
NEW.id_activo_biologico OR
           OLD.id_finca           IS DISTINCT FROM NEW.id_finca

```

```

OR
        OLD.id_patologia          IS DISTINCT FROM NEW.id_patologia
OR
        OLD.probabilidad_pct      IS DISTINCT FROM
NEW.probabilidad_pct      OR
        OLD.nivel_criticidad      IS DISTINCT FROM
NEW.nivel_criticidad      OR
        OLD.fecha_generacion     IS DISTINCT FROM
NEW.fecha_generacion
    THEN
        RAISE EXCEPTION
        '[TGR-M04-03] Los campos estructurales de la alerta son
inmutables. '
        'id_alerta_patologica=%. Solo se permite actualizar
estado_alerta, '
        'fecha_notificacion, diagnostico_confirmado,
es_verdadero, id_usuario. '
        'Requerimiento RF-67, RF-73.',
        OLD.id_alerta_patologica;
    END IF;
END IF;

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_alertas_patologicas_inmutabilidad
BEFORE UPDATE OR DELETE
ON modulo4.alertas_patologicas
FOR EACH ROW
EXECUTE FUNCTION modulo4.fn_alertas_patologicas_inmutabilidad();

```

TGR-M04-04: trg_auditorias_predicciones_inmutabili dad

Descripción	Bloquea cualquier UPDATE o DELETE sobre la tabla de auditorías de predicciones. Lanza excepción ante UPDATE o DELETE sobre <code>modulo4.auditorias_predicciones</code>.
-------------	--

Aplicado a	RF-73
Restricción origen RF	<i>"Inmutabilidad (append-only): Ningún registro de auditoría puede ser modificado o eliminado una vez persistido." — RF-73, Restricciones. CA-3: "Una vez registrado, ningún evento puede ser modificado ni eliminado (append-only)."</i>
Justificación	La tabla de auditoría es la fuente de verdad para trazabilidad regulatoria (NIC 41). Cualquier alteración de estos registros invalida la evidencia clínica y contable. Esto debe ser inviolable a nivel de motor.

```

CREATE OR REPLACE FUNCTION
modulo4.fn_auditorias_predicciones_inmutabilidad()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M04-04] Los registros de auditoría son inmutables
(append-only). '
            'No se permite eliminar id_auditoria_prediccion=%. '
            'Requerimiento RF-73, Restricción Inmutabilidad.',
            OLD.id_auditoria_prediccion;
    END IF;

    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            '[TGR-M04-04] Los registros de auditoría son inmutables
(append-only). '
            'No se permite modificar id_auditoria_prediccion=%. '
            'Requerimiento RF-73, Restricción Inmutabilidad.',
            OLD.id_auditoria_prediccion;
    END IF;

    RETURN NULL;
END;
$$;

CREATE OR REPLACE TRIGGER trg_auditorias_predicciones_inmutabilidad
BEFORE UPDATE OR DELETE
ON modulo4.auditorias_predicciones

```



```
FOR EACH ROW
EXECUTE FUNCTION modulo4.fn_auditorias_predicciones_inmutabilidad();
```

TGR-M04-05:

trg_versiones_modelos_unicidad_activ

O

Descripción	Garantiza que solo exista una versión con <code>estado_version = 'ACTIVO'</code> por <code>tipo_modelo</code>. Al activar una nueva versión, transiciona automáticamente la anterior a <code>'DEPRECADO'</code>. Cuando se inserta o actualiza una versión a <code>estado_version = 'ACTIVO'</code>, busca otras del mismo <code>tipo_modelo</code> con ese estado y las cambia a <code>'DEPRECADO'</code>. Valida además que no haya otra operación de activación concurrente (control básico).
Aplicado a	RF-69, RF-65
Restricción origen RF	<i>"Unicidad de Versión Activa: Solo puede existir una versión en estado ACTIVO por tipo_modelo en cualquier momento." — RF-69, Restricción 5. "En ningún momento pueden coexistir dos versiones ACTIVO del mismo tipo de modelo." — RF-69, Descripción.</i>
Justificación	La coexistencia de dos modelos activos causaría inferencias inconsistentes en RF-66 y conflictos en la distribución OTA (RF-70). Debe ser atómico; el backend no puede garantizarlo ante concurrencia.

```
CREATE OR REPLACE FUNCTION
modulo4.fn_versiones_modelos_unicidad_activo()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  IF NEW.estado_version = 'ACTIVO' THEN
    UPDATE modulo4.versiones_modelos
```

```

SET estado_version = 'DEPRECADO'
WHERE tipo_modelo    = NEW.tipo_modelo
    AND estado_version = 'ACTIVO'
    AND id_version_modelo <> NEW.id_version_modelo;
END IF;
RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_versiones_modelos_unicidad_activo
BEFORE INSERT OR UPDATE OF estado_version
ON modulo4.versiones_modelos
FOR EACH ROW EXECUTE FUNCTION
modulo4.fn_versiones_modelos_unicidad_activo();

```

TGR-M04-06:

trg_versiones_modelos_hash_obligatorio

Descripción	Valida que al insertar una versión de modelo, tanto hash_artefacto como hash_artefacto_sha256 sean SHA-256 válidos (64 caracteres hexadecimales). Ambos deben estar presentes. En INSERT, valida formato SHA-256 (64 hex) para hash_artefacto y hash_artefacto_sha256. Lanza excepción si alguno es nulo, vacío, de longitud incorrecta o con caracteres no hexadecimales.
Aplicado a	RF-69, RF-71
Restricción origen RF	<i>"No se registran versiones sin ambos hashes."</i> — RF-69, Restricción 8. <i>"hash_artefacto_modelo: String (SHA-256) — Longitud válida: 64 caracteres hexadecimales."</i> — RF-69, Entradas.

	RF-71, Restricción 14: <i>"RF-71 debe garantizar que el artefacto del modelo no sea modificado entre generación del hash y envío a RF-69."</i>
Justificación	Un hash ausente o con formato inválido hace imposible verificar la integridad del artefacto (RF-69, CA-9 y CA-14). Es una validación de formato puro, idónea para la BD.

```

CREATE OR REPLACE FUNCTION
modulo4.fn_versiones_modelos_hash_obligatorio()
RETURNS TRIGGER LANGUAGE plpgsql AS $$

    PROCEDURE validar_hash(p_campo TEXT, p_valor TEXT) AS $$
    BEGIN
        IF p_valor IS NULL OR trim(p_valor) = '' THEN
            RAISE EXCEPTION
                '[TGR-M04-06] El campo % es obligatorio y no puede ser nulo o
vacío. RF-69, Restricción 8.', p_campo;
        END IF;
        IF length(trim(p_valor)) <> 64 THEN
            RAISE EXCEPTION
                '[TGR-M04-06] El campo % debe tener exactamente 64 caracteres
(SHA-256). Longitud recibida: %. RF-69, Restricción 8.',
                p_campo, length(trim(p_valor));
        END IF;
        IF trim(p_valor) !~ '^[a-fA-F0-9]{64}$' THEN
            RAISE EXCEPTION
                '[TGR-M04-06] El campo % contiene caracteres no hexadecimales.
RF-69, Restricción 8.', p_campo;
        END IF;
    END;
$$

BEGIN
    PERFORM modulo4.validar_hash('hash_artefacto',
NEW.hash_artefacto);
    PERFORM modulo4.validar_hash('hash_artefacto_sha256',
NEW.hash_artefacto_sha256);
    RETURN NEW;
END;
$$;

```

Nota: PostgreSQL no soporta procedimientos anidados en funciones PL/pgSQL de esa forma. La versión correcta aplanar la lógica:

```

CREATE OR REPLACE FUNCTION
modulo4.fn_versiones_modelos_hash_obligatorio()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_hashes TEXT[] := ARRAY[
        NEW.hash_artefacto,
        NEW.hash_artefacto_sha256
    ];
    v_nombres TEXT[] := ARRAY[
        'hash_artefacto',
        'hash_artefacto_sha256'
    ];
    i INT;
BEGIN
    FOR i IN 1..array_length(v_hashes, 1) LOOP
        IF v_hashes[i] IS NULL OR trim(v_hashes[i]) = '' THEN
            RAISE EXCEPTION
                '[TGR-M04-06] El campo % es obligatorio. RF-69, Restricción 8.',
                v_nombres[i];
        END IF;
        IF length(trim(v_hashes[i])) <> 64 THEN
            RAISE EXCEPTION
                '[TGR-M04-06] El campo % debe tener 64 caracteres SHA-256.
Longitud: %. RF-69, Restricción 8.',
                v_nombres[i], length(trim(v_hashes[i]));
        END IF;
        IF trim(v_hashes[i]) !~ '^[a-fA-F0-9]{64}$' THEN
            RAISE EXCEPTION
                '[TGR-M04-06] El campo % contiene caracteres no hexadecimales.
RF-69, Restricción 8.',
                v_nombres[i];
        END IF;
    END LOOP;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_versiones_modelos_hash_obligatorio
BEFORE INSERT ON modulo4.versiones_modelos
FOR EACH ROW EXECUTE FUNCTION
modulo4.fn_versiones_modelos_hash_obligatorio();

```

TGR-M04-07:

trg_predicciones_umbral_consistencia

Descripción	Valida que umbral_usado esté en rango [50–95] y corrige automáticamente el flag supera_umbral para que sea consistente con probabilidad_pct. En INSERT, valida rango del umbral y recalcula supera_umbral automáticamente.
Aplicado a	RF-66, RF-65
Restricción origen RF	RF-65, Restricción 1: <i>"umbral_riesgo_alto debe estar entre 0.50 y 0.95."</i> RF-66, Restricción 6: <i>"Si $P(\text{riesgo_alto}) \geq \text{umbral_riesgo_alto}$, se prioriza la clase Riesgo Alto."</i>
Justificación	supera_umbral es un campo derivado. Si hay inconsistencia entre la probabilidad calculada y ese flag, las alertas y el historial diagnóstico pierden confiabilidad. La BD puede garantizar esta relación matemática simple.

```
CREATE OR REPLACE FUNCTION modulo4.fn_predicciones_umbral_consistencia()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Validar que el umbral esté en rango permitido (50.00 a 95.00)
    IF NEW.umbral_usado < 50.00 OR NEW.umbral_usado > 95.00 THEN
        RAISE EXCEPTION
            '[TGR-M04-07] El umbral_usado=% está fuera del rango
            permitido [50.00 - 95.00]. '
            'Requerimiento RF-65, Restricción 1.',
            NEW.umbral_usado;
    END IF;

    -- Corregir automáticamente la consistencia del flag supera_umbral
    IF NEW.probabilidad_pct >= NEW.umbral_usado THEN
        NEW.supera_umbral := true;
    END IF;
END;
```

```

ELSE
    NEW.supera_umbral := false;
END IF;

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_predicciones_umbral_consistencia
BEFORE INSERT
ON modulo4.predicciones
FOR EACH ROW
EXECUTE FUNCTION modulo4.fn_predicciones_umbral_consistencia();

```

TGR-M04-08:

trg_versiones_modelos_fecha_entrenamiento

Descripción	Valida que la fecha_entrenamiento de una versión de modelo no sea una fecha futura al momento de insertarla. En INSERT, compara fecha_entrenamiento con NOW() y lanza excepción si es futura.
Aplicado a	RF-69
Restricción origen RF	<i>"fecha_entrenamiento: Timestamp — No puede ser futura."</i> — RF-69, tabla de Entradas.
Justificación	Una fecha de entrenamiento futura es un dato incoherente que puede romper ordenamientos cronológicos, trazabilidad y detección de drift. La BD garantiza la consistencia temporal de forma absoluta.

```

CREATE OR REPLACE FUNCTION
modulo4.fn_versiones_modelos_fecha_entrenamiento()
RETURNS TRIGGER
LANGUAGE plpgsql

```

```

AS $$
BEGIN
    IF NEW.fecha_entrenamiento IS NOT NULL AND NEW.fecha_entrenamiento >
now() THEN
        RAISE EXCEPTION
            '[TGR-M04-08] La fecha_entrenamiento no puede ser una fecha
futura. '
            'Valor recibido: %. Requerimiento RF-69, campo
fecha_entrenamiento.',
            NEW.fecha_entrenamiento;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_versiones_modelos_fecha_entrenamiento
BEFORE INSERT
ON modulo4.versiones_modelos
FOR EACH ROW
EXECUTE FUNCTION modulo4.fn_versiones_modelos_fecha_entrenamiento();

```

TGR-M04-09:

trg_ciclos_entrenamientos_inmutabilidad_completados

Descripción	Bloquea la modificación de ciclos de entrenamiento que ya se encuentran en estado COMPLETADO o CANCELADO, permitiendo solo actualizaciones sobre ciclos en estados activos (PLANIFICADO, EN_EJECUCION). En UPDATE, verifica el estado_ciclo anterior. Si es COMPLETADO o CANCELADO, lanza excepción.
Aplicado a	RF-71, RF-73
Restricción origen RF	<i>"Inmutabilidad del Dataset: Una vez construido, el dataset no puede ser modificado durante el entrenamiento." —</i>

	RF-71, Restricción 8. Y RF-73: <i>"Ningún registro de auditoría puede ser modificado o eliminado una vez persistido."</i> RF-71 RNF-11: <i>"El historial de reentrenamiento debe ser append-only, sin posibilidad de modificación o eliminación."</i>
Justificación	Los ciclos completados o cancelados son parte del historial inmutable del proceso MLOps. Modificarlos alteraría la trazabilidad del ciclo de vida del modelo y podría generar inconsistencias con los modelos que referencia.

```

CREATE OR REPLACE FUNCTION
modulo4.fn_ciclos_entrenamientos_inmutabilidad_completados()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF OLD.estado_ciclo IN ('COMPLETADO', 'CANCELADO') THEN
        RAISE EXCEPTION
            '[TGR-M04-09] El ciclo de entrenamiento id=% en estado % no
puede ser modificado. '
            'Solo se permite modificar ciclos en estado PLANIFICADO o
EN_EJECUCION. '
            'Requerimiento RF-71 RNF-11, RF-73.',
            OLD.id_ciclo_entrenamiento,
            OLD.estado_ciclo;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER
trg_ciclos_entrenamientos_inmutabilidad_completados
BEFORE UPDATE
ON modulo4.ciclos_entrenamientos
FOR EACH ROW
EXECUTE FUNCTION
modulo4.fn_ciclos_entrenamientos_inmutabilidad_completados();

```


TGR-M04-10:

trg_ciclos_entrenamientos_no_delete

Descripción	Bloquea el DELETE sobre cualquier ciclo de entrenamiento sin excepción. Lanza excepción incondicional ante cualquier DELETE sobre modulo4.ciclos_entrenamientos.
Aplicado a	RF-71, RF-73
Restricción origen RF	RF-71 RNF-11: <i>"El historial de reentrenamiento debe ser append-only, sin posibilidad de modificación o eliminación."</i> RF-73: <i>"Ningún registro de auditoría puede ser modificado o eliminado una vez persistido."</i>
Justificación	La eliminación de un ciclo de entrenamiento destruye la trazabilidad histórica del modelo. Debe ser absolutamente imposible a nivel de motor.

```
CREATE OR REPLACE FUNCTION modulo4.fn_ciclos_entrenamientos_no_delete()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    RAISE EXCEPTION
        '[TGR-M04-10] No se permite eliminar ciclos de entrenamiento. '
        'El historial de reentrenamiento es append-only. '
        'id_ciclo_entrenamiento=%. Requerimiento RF-71 RNF-11.',
        OLD.id_ciclo_entrenamiento;
    RETURN NULL;
END;
$$;

CREATE OR REPLACE TRIGGER trg_ciclos_entrenamientos_no_delete
BEFORE DELETE
ON modulo4.ciclos_entrenamientos
FOR EACH ROW
EXECUTE FUNCTION modulo4.fn_ciclos_entrenamientos_no_delete();
```

TGR-M04-11:

trg_metricas_drift_no_delete_update

Descripción	Bloquea UPDATE y DELETE sobre métricas de drift registradas. Lanza excepción ante UPDATE o DELETE sobre <code>modulo4.metricas_drift</code> .
Aplicado a	RF-71, RF-73
Restricción origen RF	RF-73: "Inmutabilidad (append-only): Ningún registro de auditoría puede ser modificado o eliminado." RF-71 RNF-11: "El historial de reentrenamiento debe ser append-only."
Justificación	Las métricas de drift son evidencia histórica del comportamiento del modelo. Su alteración invalidaría la detección de concept drift y la justificación de reentrenamientos.

```
CREATE OR REPLACE FUNCTION modulo4.fn_metricas_drift_no_delete_update()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M04-11] Las métricas de drift son inmutables. '
            'No se permite eliminar id_metrica_drift=%. '
            'Requerimiento RF-71 RNF-11, RF-73.',
            OLD.id_metrica_drift;
    END IF;

    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            '[TGR-M04-11] Las métricas de drift son inmutables. '
            'No se permite modificar id_metrica_drift=%. '
            'Requerimiento RF-71 RNF-11, RF-73.',
            OLD.id_metrica_drift;
    END IF;
END IF;
```

```

    RETURN NULL;
END;
$$;

CREATE OR REPLACE TRIGGER trg_metricas_drift_no_delete_update
BEFORE UPDATE OR DELETE
ON modulo4.metricas_drift
FOR EACH ROW
EXECUTE FUNCTION modulo4.fn_metricas_drift_no_delete_update();

```

TGR-M04-12:

trg_datasets_inmutabilidad

Descripción	Bloquea UPDATE y DELETE sobre datasets registrados. Lanza excepción ante UPDATE o DELETE sobre <code>modulo4.datasets</code>.
Aplicado a	RF-71
Restricción origen RF	<i>"Inmutabilidad del Dataset: Una vez construido, el dataset no puede ser modificado durante el entrenamiento."</i> — RF-71, Restricción 8. RNF-08: <i>"El sistema debe garantizar: No alteración del dataset una vez generado."</i>
Justificación	Un dataset modificado después de haberse usado para entrenar un modelo rompe la reproducibilidad y la trazabilidad del proceso. La BD garantiza que el dataset registrado es exactamente el que se usó.

```

CREATE OR REPLACE FUNCTION modulo4.fn_datasets_inmutabilidad()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M04-12] Los datasets son inmutables una vez
registrados. '
    END IF;
END;

```

```

        'No se permite eliminar id_dataset=%. '
        'Requerimiento RF-71, Restricción 8, RNF-08.',
        OLD.id_dataset;
    END IF;

    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
        '[TGR-M04-12] Los datasets son inmutables una vez
registrados. '
        'No se permite modificar id_dataset=%. '
        'Requerimiento RF-71, Restricción 8, RNF-08.',
        OLD.id_dataset;
    END IF;

    RETURN NULL;
END;
$$;

CREATE OR REPLACE TRIGGER trg_datasets_inmutabilidad
BEFORE UPDATE OR DELETE
ON modulo4.datasets
FOR EACH ROW
EXECUTE FUNCTION modulo4.fn_datasets_inmutabilidad();

```

TGR-M04-13:

trg_predicciones_version_modelo_activo

Descripción	Valida que al insertar una predicción, la versión del modelo referenciada (<code>id_version_modelo</code>) tenga <code>estado_version = 'ACTIVO'</code>. En INSERT, verifica que <code>id_version_modelo</code> exista y tenga <code>estado_version = 'ACTIVO'</code>. Lanza excepción si no.
Aplicado a	RF-66, RF-69
Restricción origen RF	"El motor no puede operar sin al menos una

	<i>versión del modelo validada y activa." — RF-66, Restricción 2. RF-69: "RF-66 utiliza directamente la versión del modelo en estado ACTIVO."</i>
Justificación	Usar un modelo no activo para generar predicciones es un error de integridad operacional. La BD puede validarlo atómicamente en el mismo INSERT .

```

CREATE OR REPLACE FUNCTION
modulo4.fn_predicciones_version_modelo_activo()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_estado modulo4.enum_estado_version_modelo;
BEGIN
    SELECT estado_version INTO v_estado
    FROM modulo4.versiones_modelos
    WHERE id_version_modelo = NEW.id_version_modelo;

    IF NOT FOUND THEN
        RAISE EXCEPTION
            '[TGR-M04-13] La versión id_version_modelo=% no existe. RF-66,
Restricción 2.',
            NEW.id_version_modelo;
    END IF;

    IF v_estado <> 'ACTIVO' THEN
        RAISE EXCEPTION
            '[TGR-M04-13] Solo se generan predicciones con modelos ACTIVO. '
            'id_version_modelo=% tiene estado=%. RF-66, Restricción 2,
RF-69.',
            NEW.id_version_modelo, v_estado;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_predicciones_version_modelo_activo
BEFORE INSERT ON modulo4.predicciones
FOR EACH ROW EXECUTE FUNCTION
modulo4.fn_predicciones_version_modelo_activo();

```

TGR-M04-14:

trg_versiones_modelos_metricas_rango

Descripción	Valida que todas las métricas de desempeño estén en rango <code>[0.0 – 1.0]</code> al registrar una versión. Cubre: <code>accuracy</code>, <code>auc_roc</code>, <code>f1_score</code>, <code>precision_modelo</code>, <code>recall_modelo</code>, <code>recall_clase_riesgo_alto</code>, <code>roc_auc_score</code>. En <code>INSERT</code>, verifica que cada métrica no nula esté dentro de <code>[0.0, 1.0]</code>.
Aplicado a	RF-69
Restricción origen RF	<i>"metricas_validacion — Rango obligatorio [0.0–1.0] para métricas de escala normalizada." — RF-69, Entradas. Fase 2: "Que la estructura del JSON de métricas contenga todos los campos requeridos con valores en rango [0.0–1.0]."</i>
Justificación	Métricas fuera de rango son datos corruptos que invalidarían la evaluación automática de umbrales y comparaciones históricas.

```
CREATE OR REPLACE FUNCTION modulo4.fn_versiones_modelos_metricas_rango()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_campos TEXT[] :=
ARRAY['accuracy', 'auc_roc', 'f1_score', 'precision_modelo',
'recall_modelo', 'recall_clase_riesgo_alto', 'roc_auc_score'];
    v_valores NUMERIC[] := ARRAY[NEW.accuracy, NEW.auc_roc, NEW.f1_score,
NEW.precision_modelo, NEW.recall_modelo,
NEW.recall_clase_riesgo_alto,
NEW.roc_auc_score];
    i INT;
BEGIN
    FOR i IN 1..array_length(v_campos, 1) LOOP
```

```

    IF v_valores[i] IS NOT NULL AND (v_valores[i] < 0.0 OR v_valores[i]
> 1.0) THEN
        RAISE EXCEPTION
            '[TGR-M04-14] El campo %=% está fuera del rango [0.0-1.0].
RF-69, Entradas.',
            v_campos[i], v_valores[i];
    END IF;
END LOOP;
RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_versiones_modelos_metricas_rango
BEFORE INSERT ON modulo4.versiones_modelos
FOR EACH ROW EXECUTE FUNCTION
modulo4.fn_versiones_modelos_metricas_rango();

```

TGR-M04-15:

trg_ciclos_entrenamientos_fechas_coh erencia

Descripción	Valida que en un ciclo de entrenamiento la fecha_inicio_planificada sea anterior a la fecha_final_planificada, y que inicio_real no sea futura al momento del registro. En INSERT y UPDATE, verifica coherencia de fechas en ciclos.
Aplicado a	RF-71
Restricción origen RF	<i>"fecha_inicio_datos: Timestamp — Debe ser menor que fecha_fin_datos." — RF-71, tabla de Entradas. RF-67 Restricciones:</i> <i>"Integridad Temporal: No se pueden consultar rangos de fecha futuros ni rangos donde la fecha inicial sea mayor a la final."</i>
Justificación	Las fechas incoherentes en los ciclos de entrenamiento generarían ventanas temporales inválidas para construcción del

	dataset y harían imposible la trazabilidad cronológica.
--	---

```
CREATE OR REPLACE FUNCTION
modulo4.fn_ciclos_entrenamientos_fechas_coherencia()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.fecha_inicio_planificada >= NEW.fecha_final_planificada THEN
        RAISE EXCEPTION
            '[TGR-M04-15] La fecha_inicio_planificada debe ser anterior
a fecha_final_planificada. '
            'Recibido: inicio=%, fin=%. Requerimiento RF-71, Entradas.',
            NEW.fecha_inicio_planificada,
            NEW.fecha_final_planificada;
    END IF;

    IF NEW.inicio_real IS NOT NULL AND NEW.inicio_real > now() THEN
        RAISE EXCEPTION
            '[TGR-M04-15] El campo inicio_real no puede ser una fecha
futura. '
            'Valor recibido: %. Requerimiento RF-71.',
            NEW.inicio_real;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_ciclos_entrenamientos_fechas_coherencia
BEFORE INSERT OR UPDATE
ON modulo4.ciclos_entrenamientos
FOR EACH ROW
EXECUTE FUNCTION modulo4.fn_ciclos_entrenamientos_fechas_coherencia();
```

TGR-M04-16:

trg_observaciones_clinicas_inmutabilidad

Descripción	Bloquea DELETE sobre observaciones clínicas. Permite UPDATE solo sobre el campo observacion (nota libre), bloqueando modificaciones de datos medidos. Bloquea DELETE. En UPDATE bloquea cambios sobre campos de medición (temperatura_rectal, frecuencia_cardiaca, frecuencia_respiratoria, condicion_corporal, id_activo_biologico, fecha, fuentes_datos).
Aplicado a	RF-67, RF-73
Restricción origen RF	RF-67: <i>"Inmutabilidad: Bajo ninguna circunstancia se permite la edición o borrado de un resultado de inferencia una vez persistido."</i> Las observaciones clínicas son el origen de las predicciones; alterar su contenido comprometería la reproducibilidad.
Justificación	Las observaciones son los datos de entrada a los modelos. Si se modifican post-predicción, la trazabilidad dato→predicción queda comprometida e invalidaría auditorías.

```

CREATE OR REPLACE FUNCTION
modulo4.fn_observaciones_clinicas_inmutabilidad()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M04-16] No se permite eliminar observaciones clínicas.'
            ,
            'id_observacion_clinica=%. Requerimiento RF-67, RF-73.',
            OLD.id_observacion_clinica;
    END IF;

    IF TG_OP = 'UPDATE' THEN
        IF OLD.id_activo_biologico IS DISTINCT FROM
NEW.id_activo_biologico OR
        OLD.fecha IS DISTINCT FROM NEW.fecha
OR
        OLD.temperatura_rectal IS DISTINCT FROM

```

```

NEW.temperatura_rectal      OR
      OLD.frecuencia_cardiaca      IS DISTINCT FROM
NEW.frecuencia_cardiaca      OR
      OLD.frecuencia_respiratoria IS DISTINCT FROM
NEW.frecuencia_respiratoria OR
      OLD.condicion_corporal      IS DISTINCT FROM
NEW.condicion_corporal      OR
      OLD.fuente_datos            IS DISTINCT FROM NEW.fuente_datos
THEN
    RAISE EXCEPTION
        '[TGR-M04-16] Los datos clínicos medidos son inmutables.
        '
        'id_observacion_clinica=%. Solo se permite modificar el
campo observacion. '
        'Requerimiento RF-67, RF-73.',
        OLD.id_observacion_clinica;
    END IF;
END IF;

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_observaciones_clinicas_inmutabilidad
BEFORE UPDATE OR DELETE
ON modulo4.observaciones_clinicas
FOR EACH ROW
EXECUTE FUNCTION modulo4.fn_observaciones_clinicas_inmutabilidad();

```

TGR-M04-17:

trg_resultados_inferencia_inmutabilidad

Descripción	
	Impide UPDATE o DELETE sobre modulo4.resultados_inferencia. Esta es la tabla principal de RF-66 en el DDL actual, distinta de predicciones. Lanza excepción incondicional ante cualquier UPDATE o DELETE.

Aplicado a	RF-66, RF-67
Restricción origen RF	RF-66, Restricción 5: <i>"Los resultados de inferencia son inmutables una vez persistidos."</i> RF-67: <i>"Bajo ninguna circunstancia se permite la edición o borrado de un resultado de inferencia una vez persistido."</i>
Justificación	resultados_inferencia es la tabla central del motor de inferencia en tiempo real (RF-66). Es la fuente que alimenta alertas (RF-57), historial (RF-67) y retroalimentación (RF-72). Su inmutabilidad es un pilar de integridad del sistema que la BD debe garantizar independientemente del backend.

```

CREATE OR REPLACE FUNCTION
modulo4.fn_resultados_inferencia_inmutabilidad()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M04-17] Los resultados de inferencia son inmutables
            (append-only). '
            'No se permite eliminar id_resultado_inferencia=%. RF-66,
            Restricción 5, RF-67.',
            OLD.id_resultado_inferencia;
    END IF;
    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            '[TGR-M04-17] Los resultados de inferencia son inmutables
            (append-only). '
            'No se permite modificar id_resultado_inferencia=%. RF-66,
            Restricción 5, RF-67.',
            OLD.id_resultado_inferencia;
    END IF;
    RETURN NULL;
END;
$$;

CREATE OR REPLACE TRIGGER trg_resultados_inferencia_inmutabilidad
BEFORE UPDATE OR DELETE ON modulo4.resultados_inferencia
FOR EACH ROW EXECUTE FUNCTION
modulo4.fn_resultados_inferencia_inmutabilidad();

```

TGR-M04-18:

trg_retroalimentaciones_clinicas_inmutabilidad

Descripción	Impide UPDATE o DELETE sobre modulo4.retroalimentaciones_clinicas. Lanza excepción incondicional ante UPDATE o DELETE.
Aplicado a	RF-72, RF-73
Restricción origen RF	RF-72, Restricciones: <i>"Una vez registrada, la retroalimentación no puede ser editada ni eliminada, garantizando integridad histórica."</i> RF-72, RNF-04: <i>"Los registros de retroalimentación no pueden ser modificados ni eliminados después de su creación."</i>
Justificación	La retroalimentación clínica es la etiqueta verificada (ground truth) que alimenta el reentrenamiento (RF-71). Modificarla post-registro alteraría el conjunto de entrenamiento de forma no trazable, lo cual es inaceptable para un sistema de IA médica.

```
CREATE OR REPLACE FUNCTION
modulo4.fn_retroalimentaciones_clinicas_inmutabilidad()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M04-18] Las retroalimentaciones clínicas son inmutables
            (append-only). '
            'No se permite eliminar id_retroalimentacion=%. RF-72, RNF-04.',
            OLD.id_retroalimentacion;
    END IF;
    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            '[TGR-M04-18] Las retroalimentaciones clínicas son inmutables
```

```

(append-only). '
    'No se permite modificar id_retroalimentacion=%. RF-72, RNF-04.',
    OLD.id_retroalimentacion;
END IF;
RETURN NULL;
END;
$$;

CREATE OR REPLACE TRIGGER trg_retroalimentaciones_clinicas_inmutabilidad
BEFORE UPDATE OR DELETE ON modulo4.retroalimentaciones_clinicas
FOR EACH ROW EXECUTE FUNCTION
modulo4.fn_retroalimentaciones_clinicas_inmutabilidad();

```

TGR-M04-19:

trg_retroalimentaciones_diagnostico_obligatorio

Descripción	Valida que cuando <code>estado_retroalimentacion</code> es <code>PARCIAL</code> o <code>INCORRECTO</code>, el array <code>diagnosticos_reales</code> no sea nulo ni vacío, y que tenga entre 1 y 3 elementos. En <code>INSERT</code>, si el estado es <code>PARCIAL</code> o <code>INCORRECTO</code>, verifica que <code>diagnosticos_reales</code> no sea nulo, tenga al menos 1 elemento y no supere 3.
Aplicado a	RF-72
Restricción origen RF	RF-72, Restricciones: "El diagnóstico clínico real es obligatorio cuando el estado es <code>PARCIAL</code> o <code>INCORRECTO</code>." RF-72, Entradas: "Lista de 1–3 IDs de patologías. Obligatorio si <code>estado_retroalimentacion</code> es <code>PARCIAL</code> o <code>INCORRECTO</code>."
Justificación	Es una regla de negocio con dominio cerrado y condición booleana simple. La BD puede evaluarla de forma atómica en el <code>INSERT</code>, evitando retroalimentaciones incompletas que contaminan el dataset de

	reentrenamiento.
--	------------------

```

CREATE OR REPLACE FUNCTION
modulo4.fn_retroalimentaciones_diagnostico_obligatorio()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.estado_retroalimentacion IN ('PARCIAL', 'INCORRECTO') THEN
        IF NEW.diagnosticos_reales IS NULL OR
array_length(NEW.diagnosticos_reales, 1) IS NULL THEN
            RAISE EXCEPTION
                '[TGR-M04-19] diagnosticos_reales es obligatorio cuando
estado_retroalimentacion=%.'
                'RF-72, Restricciones: obligatoriedad condicional.',
                NEW.estado_retroalimentacion;
        END IF;
        IF array_length(NEW.diagnosticos_reales, 1) > 3 THEN
            RAISE EXCEPTION
                '[TGR-M04-19] diagnosticos_reales no puede tener más de 3
patologías.'
                'Recibido: % elementos. RF-72, Entradas.',
                array_length(NEW.diagnosticos_reales, 1);
        END IF;
    END IF;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER
trg_retroalimentaciones_diagnostico_obligatorio
BEFORE INSERT ON modulo4.retroalimentaciones_clinicas
FOR EACH ROW EXECUTE FUNCTION
modulo4.fn_retroalimentaciones_diagnostico_obligatorio();

```

TGR-M04-20:

trg_retroalimentaciones_timestamp_coherencia

Descripción	Valida que el timestamp de la retroalimentación sea posterior al fecha_inferencia del resultado de inferencia al que está vinculada.
-------------	--

	En INSERT, consulta resultados_inferencia.fecha_inferencia para el id_resultado_inferencia referenciado y verifica que fecha_retroalimentacion > fecha_inferencia.
Aplicado a	RF-72
Restricción origen RF	RF-72, Restricciones: <i>"El timestamp de la retroalimentación debe ser posterior al timestamp del resultado de inferencia evaluado."</i> Flujo alternativo E7: <i>"HTTP 422 — La fecha de retroalimentación es inválida."</i>
Justificación	Es una validación de consistencia temporal entre dos tablas. La BD puede hacerla atómicamente en el mismo INSERT sin requerir lógica adicional en el backend, y garantiza que ningún registro inválido persista aunque haya bugs en la capa de aplicación.

```

CREATE OR REPLACE FUNCTION
modulo4.fn_retroalimentaciones_timestamp_coherencia()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_fecha_inferencia TIMESTAMP WITH TIME ZONE;
BEGIN
    SELECT fecha_inferencia INTO v_fecha_inferencia
    FROM modulo4.resultados_inferencia
    WHERE id_resultado_inferencia = NEW.id_resultado_inferencia;

    IF NOT FOUND THEN
        RAISE EXCEPTION
            '[TGR-M04-20] El resultado de inferencia id=% no existe. RF-72,
            Flujo E1.',
            NEW.id_resultado_inferencia;
    END IF;

    IF NEW.fecha_retroalimentacion <= v_fecha_inferencia THEN
        RAISE EXCEPTION
            '[TGR-M04-20] fecha_retroalimentacion (%) debe ser posterior a
            fecha_inferencia (%). '

```

```

        'RF-72, Restricciones: Consistencia temporal, Flujo E7.',
        NEW.fecha_retroalimentacion, v_fecha_inferencia;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_retroalimentaciones_timestamp_coherencia
BEFORE INSERT ON modulo4.retroalimentaciones_clinicas
FOR EACH ROW EXECUTE FUNCTION
modulo4.fn_retroalimentaciones_timestamp_coherencia();

```

TGR-M04-21:

trg_eventos_auditoria_m04_inmutabilidad

Descripción	Impide UPDATE o DELETE sobre modulo4.eventos_auditoria_m04. Lanza excepción incondicional ante UPDATE o DELETE.
Aplicado a	RF-73
Restricción origen RF	RF-73, Restricciones: "Inmutabilidad (append-only): Ningún registro de auditoría puede ser modificado o eliminado una vez persistido." CA-3: "Una vez registrado, ningún evento puede ser modificado ni eliminado (append-only)."
Justificación	eventos_auditoria_m04 es el registro central de trazabilidad del módulo (RF-64 a RF-72). La integridad de estos registros es requerida para cumplimiento regulatorio (NIC 41) y gobernanza de IA. No puede depender solo de políticas de aplicación.

```

CREATE OR REPLACE FUNCTION
modulo4.fn_eventos_auditoria_m04_inmutabilidad()

```



```

RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M04-21] Los eventos de auditoría M04 son inmutables
(append-only). '
            'No se permite eliminar id_evento=%. RF-73, CA-3.',
            OLD.id_evento;
    END IF;
    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            '[TGR-M04-21] Los eventos de auditoría M04 son inmutables
(append-only). '
            'No se permite modificar id_evento=%. RF-73, CA-3.',
            OLD.id_evento;
    END IF;
    RETURN NULL;
END;
$$;

CREATE OR REPLACE TRIGGER trg_eventos_auditoria_m04_inmutabilidad
BEFORE UPDATE OR DELETE ON modulo4.eventos_auditoria_m04
FOR EACH ROW EXECUTE FUNCTION
modulo4.fn_eventos_auditoria_m04_inmutabilidad();

```

TGR-M04-22:

trg_eventos_auditoria_m04_fuente_exclusiva

Descripción	Valida que todo evento de auditoría tenga exactamente una fuente identificada: si <code>tipo_actor = 'USUARIO'</code> entonces <code>id_usuario</code> es obligatorio e <code>id_sistema</code> debe ser nulo; si <code>tipo_actor = 'SISTEMA'</code> o <code>'DISPOSITIVO_EDGE'</code> entonces <code>id_sistema</code> es obligatorio e <code>id_usuario</code> debe ser nulo. No se permiten ambos nulos simultáneamente. En <code>INSERT</code>, verifica la regla de
-------------	---

	exclusividad según tipo_actor . Lanza excepción si la combinación es inválida.
Aplicado a	RF-73
Restricción origen RF	RF-73, Restricciones: <i>"Exclusividad entre id_usuario e id_sistema: Todo evento debe tener exactamente una fuente identificada."</i> CA-13: <i>"El sistema rechaza cualquier evento donde tanto id_usuario como id_sistema sean nulos simultáneamente."</i>
Justificación	La exclusividad de fuente es una invariante de integridad referencial entre campos del mismo registro. Es una validación sin efectos secundarios, determinista y pura: idónea para la BD.

```

CREATE OR REPLACE FUNCTION
modulo4.fn_eventos_auditoria_m04_fuente_exclusiva()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.tipo_actor = 'USUARIO' THEN
        IF NEW.id_usuario IS NULL THEN
            RAISE EXCEPTION
                '[TGR-M04-22] Para tipo_actor=USUARIO, id_usuario es
obligatorio. RF-73, CA-13.';
        END IF;
        IF NEW.id_sistema IS NOT NULL THEN
            RAISE EXCEPTION
                '[TGR-M04-22] Para tipo_actor=USUARIO, id_sistema debe ser nulo.
RF-73, CA-13.';
        END IF;
    ELSIF NEW.tipo_actor IN ('SISTEMA', 'DISPOSITIVO_EDGE') THEN
        IF NEW.id_sistema IS NULL THEN
            RAISE EXCEPTION
                '[TGR-M04-22] Para tipo_actor=%, id_sistema es obligatorio.
RF-73, CA-13.',
                NEW.tipo_actor;
        END IF;
        IF NEW.id_usuario IS NOT NULL THEN
            RAISE EXCEPTION
                '[TGR-M04-22] Para tipo_actor=%, id_usuario debe ser nulo.
RF-73, CA-13.',
                NEW.tipo_actor;
        END IF;
    ELSE

```

```

        IF NEW.id_usuario IS NULL AND NEW.id_sistema IS NULL THEN
            RAISE EXCEPTION
                '[TGR-M04-22] Todo evento debe tener al menos una fuente
identificada '
                '(id_usuario o id_sistema). RF-73, Restricciones.';
        END IF;
    END IF;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_eventos_auditoria_m04_fuente_exclusiva
BEFORE INSERT ON modulo4.eventos_auditoria_m04
FOR EACH ROW EXECUTE FUNCTION
modulo4.fn_eventos_auditoria_m04_fuente_exclusiva();

```

TGR-M04-23:

trg_historial_catalogo_patologias_inmutabilidad

Descripción	Impide UPDATE o DELETE sobre modulo4.historial_catalogo_patologias. Lanza excepción incondicional ante UPDATE o DELETE.
Aplicado a	RF-64, RF-73
Restricción origen RF	RF-64, Restricción 11: "No se permite eliminar físicamente patologías; solo eliminación lógica." RF-64, Restricción 10: "Toda modificación de patologías no base genera una nueva version_catalogo." RF-73: "Ningún registro de auditoría puede ser modificado o eliminado."
Justificación	El historial del catálogo de patologías es la trazabilidad de qué versión del catálogo existía en cada momento. RF-69 y RF-71 dependen de la consistencia de este historial para saber con qué clases objetivo fue entrenado cada modelo.

```

CREATE OR REPLACE FUNCTION
modulo4.fn_historial_catalogo_patologias_inmutabilidad()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M04-23] El historial del catálogo de patologías es inmutable
            (append-only). '
            'No se permite eliminar id_historial_catalogo=%. RF-64, RF-73.',
            OLD.id_historial_catalogo;
    END IF;
    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            '[TGR-M04-23] El historial del catálogo de patologías es inmutable
            (append-only). '
            'No se permite modificar id_historial_catalogo=%. RF-64, RF-73.',
            OLD.id_historial_catalogo;
    END IF;
    RETURN NULL;
END;
$$;

CREATE OR REPLACE TRIGGER
trg_historial_catalogo_patologias_inmutabilidad
BEFORE UPDATE OR DELETE ON modulo4.historial_catalogo_patologias
FOR EACH ROW EXECUTE FUNCTION
modulo4.fn_historial_catalogo_patologias_inmutabilidad();

```

TGR-M04-24:

trg_historial_estados_modelos_inmutabilidad

Descripción	Impide UPDATE o DELETE sobre modulo4.historial_estados_modelos. Lanza excepción incondicional ante UPDATE o DELETE.
Aplicado a	RF-69, RF-73

Restricción origen RF	RF-69, Restricción 7: <i>"Inmutabilidad del Historial: Todas las transiciones se almacenan en esquema append-only."</i> RF-73, CA-3.
Justificación	Las transiciones de estado de modelos son la evidencia auditable del ciclo de vida de la IA. Cualquier alteración comprometería la trazabilidad requerida por RF-73 y el cumplimiento de NIC 41.

```

CREATE OR REPLACE FUNCTION
modulo4.fn_historial_estados_modelos_inmutabilidad()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M04-24] El historial de estados de modelos es inmutable
            (append-only). '
            'No se permite eliminar id_historial_estado_modelo=%. RF-69,
            RF-73.',
            OLD.id_historial_estado_modelo;
    END IF;
    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            '[TGR-M04-24] El historial de estados de modelos es inmutable
            (append-only). '
            'No se permite modificar id_historial_estado_modelo=%. RF-69,
            RF-73.',
            OLD.id_historial_estado_modelo;
    END IF;
    RETURN NULL;
END;
$$;

CREATE OR REPLACE TRIGGER trg_historial_estados_modelos_inmutabilidad
BEFORE UPDATE OR DELETE ON modulo4.historial_estados_modelos
FOR EACH ROW EXECUTE FUNCTION
modulo4.fn_historial_estados_modelos_inmutabilidad();

```

TGR-M04-25:

trg_historial_diagnostico_eventos_inmutabilidad

Descripción	Impide UPDATE o DELETE sobre modulo4.historial_diagnostico_eventos. Lanza excepción incondicional ante UPDATE o DELETE.
Aplicado a	RF-67, RF-73
Restricción origen RF	RF-67, Restricciones: <i>"Inmutabilidad: Bajo ninguna circunstancia se permite la edición o borrado de un resultado de inferencia una vez persistido."</i> RF-67, Restricciones: <i>"Unicidad: No se permiten eventos duplicados en el historial."</i>
Justificación	El historial de eventos diagnósticos es el registro cronológico de la salud del activo biológico. Es consumido por M06 para valoración NIC 41 y por auditores. Su integridad debe ser absoluta.

```
CREATE OR REPLACE FUNCTION
modulo4.fn_historial_diagnostico_eventos_inmutabilidad()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M04-25] El historial de eventos diagnósticos es inmutable
            (append-only). '
            'No se permite eliminar id_evento=%. RF-67.',
            OLD.id_evento;
    END IF;
    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            '[TGR-M04-25] El historial de eventos diagnósticos es inmutable
            (append-only). '
            'No se permite modificar id_evento=%. RF-67.',
```

```

        OLD.id_evento;
    END IF;
    RETURN NULL;
END;
$$;

CREATE OR REPLACE TRIGGER
trg_historial_diagnostico_eventos_inmutabilidad
BEFORE UPDATE OR DELETE ON modulo4.historial_diagnostico_eventos
FOR EACH ROW EXECUTE FUNCTION
modulo4.fn_historial_diagnostico_eventos_inmutabilidad();

```

TGR-M04-26:

trg_versiones_modelos_transicion_estado_valida

Descripción	Valida que las transiciones de estado_version sigan la máquina de estados cerrada definida en RF-69. Impide transiciones inválidas y bloquea la reactivación de modelos DEPRECADO o RECHAZADO. En UPDATE de estado_version, verifica que la transición (OLD.estado_version → NEW.estado_version) esté en la lista de transiciones permitidas. Lanza excepción si no.
Aplicado a	RF-69
Restricción origen RF	RF-69, Descripción: "Transiciones válidas: <i>EN_VALIDACION</i>→<i>APROBADO/RECHAZADO</i> (automático); <i>APROBADO</i>→<i>ACTIVO</i> (manual); <i>ACTIVO</i>→<i>DEPRECADO</i> (automático). No se permiten transiciones fuera de este flujo. Un modelo <i>DEPRECADO</i> o <i>RECHAZADO</i> no puede reactivarse bajo ninguna circunstancia." RF-69, Restricción 6.

Justificación	La máquina de estados es una invariante de dominio crítica. Un error de backend podría intentar reactivar un modelo deprecado. La BD la hace inviolable sin importar la capa que intente la operación.
----------------------	--

```

CREATE OR REPLACE FUNCTION
modulo4.fn_versiones_modelos_transicion_estado_valida()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF OLD.estado_version = NEW.estado_version THEN
        RETURN NEW;
    END IF;

    IF OLD.estado_version IN ('DEPRECADO', 'RECHAZADO') THEN
        RAISE EXCEPTION
            '[TGR-M04-26] Los modelos en estado % no pueden cambiar de estado.
            '
            'id_version_modelo=%. RF-69, Restricción 6.',
            OLD.estado_version, OLD.id_version_modelo;
    END IF;

    IF NOT (
        (OLD.estado_version = 'EN_VALIDACION' AND NEW.estado_version IN
        ('APROBADO', 'RECHAZADO'))
        OR
        (OLD.estado_version = 'APROBADO' AND NEW.estado_version IN
        ('ACTIVO', 'DEPRECADO'))
        OR
        (OLD.estado_version = 'ACTIVO' AND NEW.estado_version =
        'DEPRECADO')
    ) THEN
        RAISE EXCEPTION
            '[TGR-M04-26] Transición de estado inválida: %→%. '
            'id_version_modelo=%. RF-69, Descripción: Transiciones válidas.',
            OLD.estado_version, NEW.estado_version, OLD.id_version_modelo;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_versiones_modelos_transicion_estado_valida
BEFORE UPDATE OF estado_version ON modulo4.versiones_modelos
FOR EACH ROW EXECUTE FUNCTION

```



```
modulo4.fn_versiones_modelos_transicion_estado_valida();
```

Nota: Este trigger coexiste con TGR-M04-01 (inmutabilidad total). El orden de ejecución en PostgreSQL para triggers **BEFORE** del mismo tipo es alfabético por nombre.

trg_versiones_modelos_transicion_estado_valida se ejecuta antes de **trg_versiones_modelos_inmutabilidad** por orden alfabético, lo que genera conflicto: TGR-M04-01 bloqueará cualquier **UPDATE** antes de que TGR-M04-26 pueda validar la transición. **Solución:** TGR-M04-01 debe modificarse para permitir **UPDATE** exclusivamente del campo **estado_version** (y solo ese campo), bloqueando todo lo demás. Se documenta a continuación como corrección.

TGR-M04-27:

trg_versiones_modelos_notas_requeridas_activacion

Descripción	Bloquea la activación (estado_version = 'ACTIVO') de una versión de modelo si notas_validacion es nulo o está vacío. En UPDATE de estado_version, cuando el nuevo estado es 'ACTIVO', verifica que notas_validacion no sea nulo ni vacío.
Aplicado a	RF-69
Restricción origen RF	RF-69, CA-13: "El sistema no permite activar una versión sin que el Veterinario o Administrador haya registrado notas_validacion con contenido no vacío." RF-69, Fase 4: "El Veterinario registra notas_validacion con su evaluación clínica del modelo."
Justificación	Es una precondition de negocio para la activación. La BD puede verificarla de forma atómica en el mismo UPDATE que intenta la activación, garantizando que nunca un modelo entre a producción sin revisión clínica documentada.

```

CREATE OR REPLACE FUNCTION
modulo4.fn_versiones_modelos_notas_requeridas_activacion()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.estado_version = 'ACTIVO' AND OLD.estado_version <> 'ACTIVO'
THEN
        IF NEW.notas_validacion IS NULL OR trim(NEW.notas_validacion) = ''
THEN
            RAISE EXCEPTION
                '[TGR-M04-27] No se puede activar la versión id=% sin
notas_validacion. '
                'El Veterinario o Administrador debe registrar la evaluación
clínica antes de activar. '
                'RF-69, CA-13.',
                OLD.id_version_modelo;
        END IF;
    END IF;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER
trg_versiones_modelos_notas_requeridas_activacion
BEFORE UPDATE OF estado_version ON modulo4.versiones_modelos
FOR EACH ROW EXECUTE FUNCTION
modulo4.fn_versiones_modelos_notas_requeridas_activacion();

```

TGR-M04-28:

trg_configuraciones_motor_umbrales_ coherencia

Descripción	Valida que umbral_alerta_critica >= umbral_riesgo_alto, que ambos estén en rango [0.50, 0.95], y que ventana_temporal_min esté entre 5 y 15. En INSERT y UPDATE, valida rangos de umbrales y ventana, y la relación umbral_alerta_critica >= umbral_riesgo_alto.
-------------	---

Aplicado a	RF-65
Restricción origen RF	RF-65, Restricción 6: "Se debe cumplir que $umbral_alerta_critica \geq umbral_riesgo_alto$." RF-65, Restricción 1: "umbral_riesgo_alto debe estar entre 0.50 y 0.95." RF-65, Restricción 2: "ventana_temporal_minutos debe estar entre 5 y 15." Flujo alterno E1, E4.
Justificación	Son invariantes matemáticas entre columnas del mismo registro, sin dependencias externas. La BD las puede verificar de forma atómica, evitando configuraciones incoherentes que afecten directamente el comportamiento del motor de inferencia en producción.

```

CREATE OR REPLACE FUNCTION
modulo4.fn_configuraciones_motor_umbrales_coherencia()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.umbral_riesgo_alto < 0.500 OR NEW.umbral_riesgo_alto > 0.950
    THEN
        RAISE EXCEPTION
            '[TGR-M04-28] umbral_riesgo_alto=% fuera del rango [0.50-0.95].
RF-65, Restricción 1.',
            NEW.umbral_riesgo_alto;
    END IF;

    IF NEW.umbral_alerta_critica < 0.500 OR NEW.umbral_alerta_critica >
0.950 THEN
        RAISE EXCEPTION
            '[TGR-M04-28] umbral_alerta_critica=% fuera del rango [0.50-0.95].
RF-65, Restricción 1.',
            NEW.umbral_alerta_critica;
    END IF;

    IF NEW.umbral_alerta_critica < NEW.umbral_riesgo_alto THEN
        RAISE EXCEPTION
            '[TGR-M04-28] umbral_alerta_critica (%) debe ser >=
umbral_riesgo_alto (%). '
            'RF-65, Restricción 6, Flujo E4.',
            NEW.umbral_alerta_critica, NEW.umbral_riesgo_alto;
    END IF;

    IF NEW.ventana_temporal_min < 5 OR NEW.ventana_temporal_min > 15 THEN

```

```

        RAISE EXCEPTION
        '[TGR-M04-28] ventana_temporal_min=% fuera del rango [5-15]
minutos. RF-65, Restricción 2.',
        NEW.ventana_temporal_min;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_configuraciones_motor_umbrales_coherencia
BEFORE INSERT OR UPDATE ON modulo4.configuraciones_motor_ia
FOR EACH ROW EXECUTE FUNCTION
modulo4.fn_configuraciones_motor_umbrales_coherencia();

```

TGR-M04-29:

trg_configuraciones_motor_pesos_con tagio

Descripción	Valida que los pesos del modelo de contagio (w_factor_sanitario + w_factor_ambiental + w_factor_densidad) sumen exactamente 1.0, con una tolerancia de ±0.001 por precisión numérica. En INSERT y UPDATE, verifica que cada peso esté en [0.0, 1.0] y que la suma de los tres sea 1.0 ± 0.001.
Aplicado a	RF-68, RF-65
Restricción origen RF	RF-68, Descripción: "<i>W_{fs} + W_{fa} + W_{fd} = 1.0. Los pesos deben estar en el rango [0.0, 1.0]. Se deben validar antes de ejecutar el modelo.</i>" RF-68, Proceso 5.2: "<i>Se valida que la suma de los pesos sea igual a 1.0 antes del cálculo. Si los pesos no cumplen la validación → No se ejecuta el modelo → Se registra evento: ERROR_CONFIGURACION_PESOS.</i>"

Justificación	La suma de pesos es una invariante matemática del modelo de contagio. Si no suma 1.0, el modelo $P_{\text{contagio}} = W_{fs} \cdot F_s + W_{fa} \cdot F_a + W_{fd} \cdot F_d$ produce resultados fuera del rango $[0, 1]$ y las clasificaciones de riesgo serían incorrectas. La BD garantiza que nunca persista una configuración inválida.
----------------------	--

```

CREATE OR REPLACE FUNCTION
modulo4.fn_configuraciones_motor_pesos_contagio()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_suma NUMERIC;
BEGIN
    IF NEW.w_factor_sanitario < 0.0 OR NEW.w_factor_sanitario > 1.0 THEN
        RAISE EXCEPTION
            '[TGR-M04-29] w_factor_sanitario=% fuera del rango [0.0-1.0].
RF-68.',
            NEW.w_factor_sanitario;
    END IF;
    IF NEW.w_factor_ambiental < 0.0 OR NEW.w_factor_ambiental > 1.0 THEN
        RAISE EXCEPTION
            '[TGR-M04-29] w_factor_ambiental=% fuera del rango [0.0-1.0].
RF-68.',
            NEW.w_factor_ambiental;
    END IF;
    IF NEW.w_factor_densidad < 0.0 OR NEW.w_factor_densidad > 1.0 THEN
        RAISE EXCEPTION
            '[TGR-M04-29] w_factor_densidad=% fuera del rango [0.0-1.0].
RF-68.',
            NEW.w_factor_densidad;
    END IF;

    v_suma := NEW.w_factor_sanitario + NEW.w_factor_ambiental +
NEW.w_factor_densidad;

    IF abs(v_suma - 1.0) > 0.001 THEN
        RAISE EXCEPTION
            '[TGR-M04-29] La suma de pesos del modelo de contagio debe ser
1.0. '
            'Suma actual: % (W_fs=%, W_fa=%, W_fd=%). RF-68, Descripción.',
            v_suma, NEW.w_factor_sanitario, NEW.w_factor_ambiental,
NEW.w_factor_densidad;
    END IF;

```

```

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_configuraciones_motor_pesos_contagio
BEFORE INSERT OR UPDATE ON modulo4.configuraciones_motor_ia
FOR EACH ROW EXECUTE FUNCTION
modulo4.fn_configuraciones_motor_pesos_contagio();

```

TGR-M04-30:

trg_despliegues_ota_inmutabilidad_completados

Descripción	Bloquea UPDATE sobre despliegues OTA que ya tienen estado EXITOSO o FALLIDO (terminales). Permite actualización solo sobre despliegues en estados activos (PENDIENTE, EN_PROCESO). Bloquea DELETE incondicional. Bloquea DELETE siempre. Bloquea UPDATE si OLD.estado_despliegue IN ('EXITOSO', 'FALLIDO').
Aplicado a	RF-70, RF-73
Restricción origen RF	RF-70, Postcondiciones: "Trazabilidad completa del proceso OTA... el proceso de despliegue queda registrado en el sistema de auditoría." RF-73: "Ningún registro de auditoría puede ser modificado o eliminado." RF-70, CA-9: "Todo el proceso de despliegue OTA queda registrado en auditoría con trazabilidad completa por dispositivo."
Justificación	Los despliegues OTA completados son registros históricos de qué versión de modelo llegó a cada dispositivo. Modificarlos comprometería la trazabilidad de qué modelo está ejecutando cada nodo Edge, con implicaciones directas sobre la

	validez de las inferencias generadas (RF-66).
--	--

```
CREATE OR REPLACE FUNCTION
modulo4.fn_despliegues_ota_inmutabilidad_completados()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M04-30] No se permite eliminar registros de despliegue OTA.
            '
            'id_despliegue_ota=%. RF-70, RF-73.',
            OLD.id_despliegue_ota;
    END IF;

    IF TG_OP = 'UPDATE' AND OLD.estado_despliegue IN ('EXITOSO',
'FALLIDO') THEN
        RAISE EXCEPTION
            '[TGR-M04-30] Los despliegues OTA en estado % son inmutables. '
            'id_despliegue_ota=%. RF-70, RF-73.',
            OLD.estado_despliegue, OLD.id_despliegue_ota;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_despliegues_ota_inmutabilidad_completados
BEFORE UPDATE OR DELETE ON modulo4.despliegues_ota
FOR EACH ROW EXECUTE FUNCTION
modulo4.fn_despliegues_ota_inmutabilidad_completados();
```